



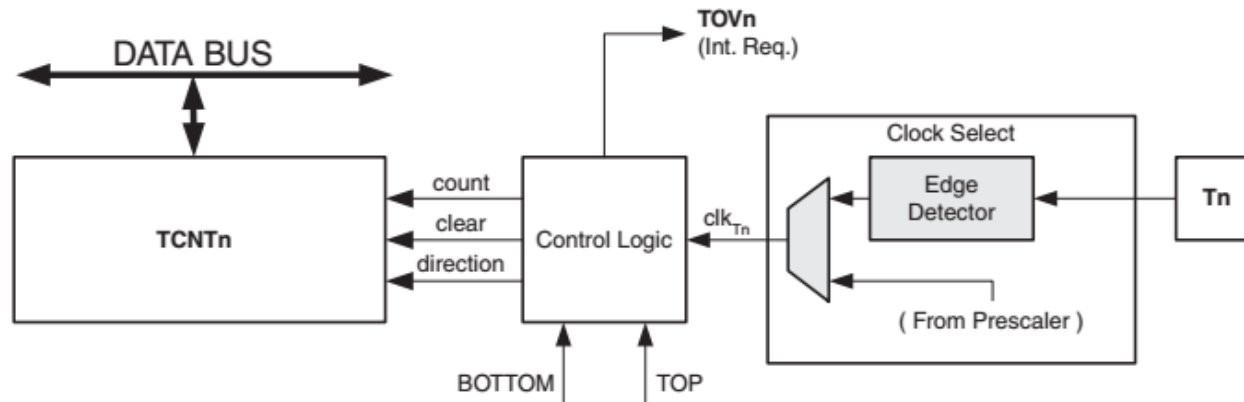
CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

Facultad de Ingeniería
UNLP

PERIFERICO TIMER
(AVR)

Ing. José Juárez

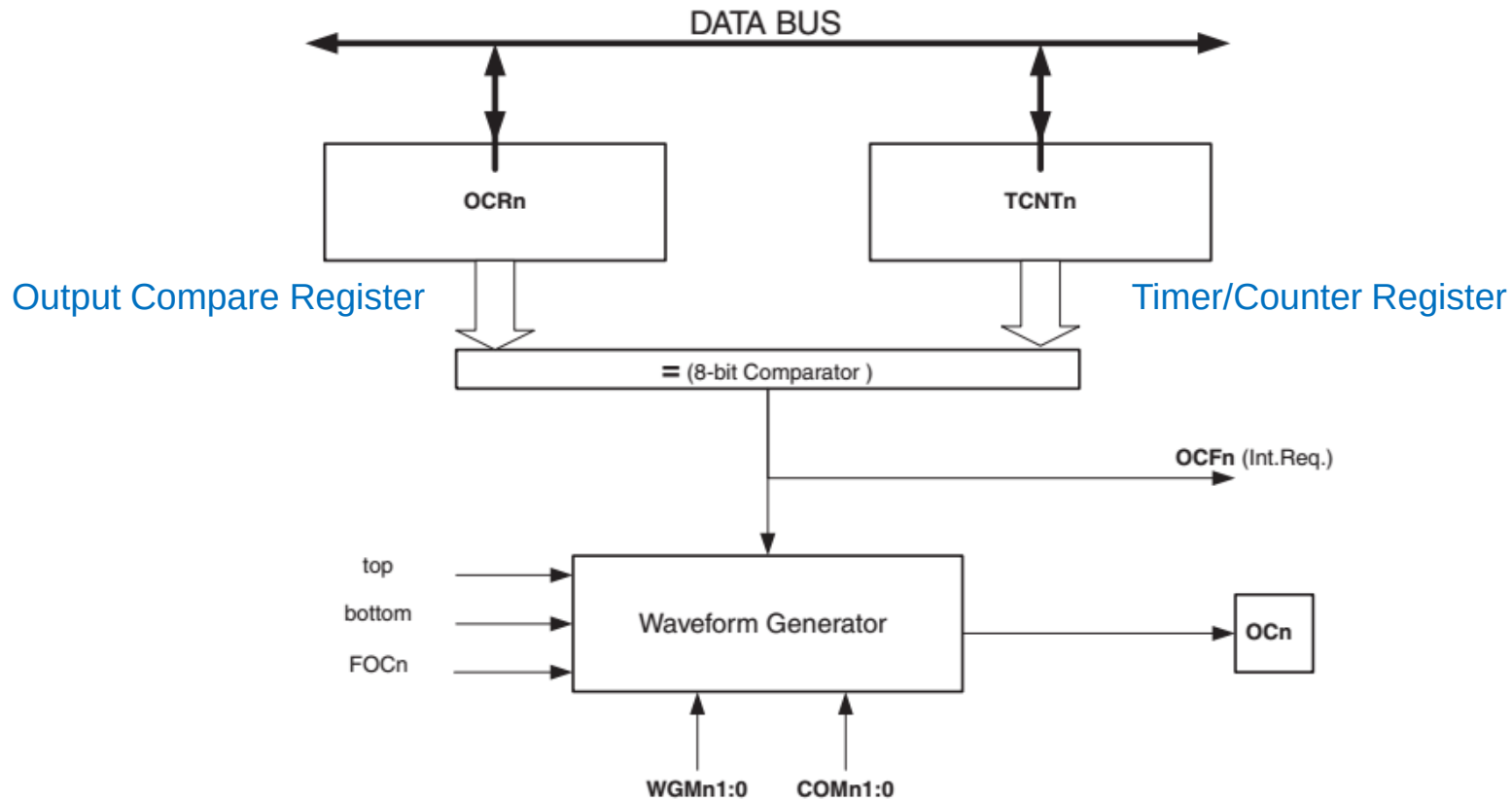
Sistema básico de temporización



- El corazón del sistema es un registro contador, llamado TCNT “Timer Counter”
- El contador suma 1 (o resta 1 según dirección de conteo) cada vez que en su entrada hay un flanco del reloj (count).
- La señal de reloj puede provenir desde el Prescaler o por terminal externo (Tn)
 - Si contamos pulsos del reloj del MCU (From Prescaler) tenemos una base de tiempo con prestaciones conocidas.
 - Si contamos pulsos externos de una señal “desconocida”, estamos contando cuantas veces ocurrió un flanco en el terminal Tn.
- La capacidad de conteo depende del número de bits del contador, por ejemplo con 16 bits puede contar hasta 65536 y cuando llega al máximo valor 0xFFFF se reinicia (clear) automáticamente generando un aviso de desborde u overflow (flag TOVn).
- BOTTOM es el valor inicial (por defecto 0)
- TOP es el valor final por lo tanto el contador es módulo MOD=TOP+1

Timer con generador de señales

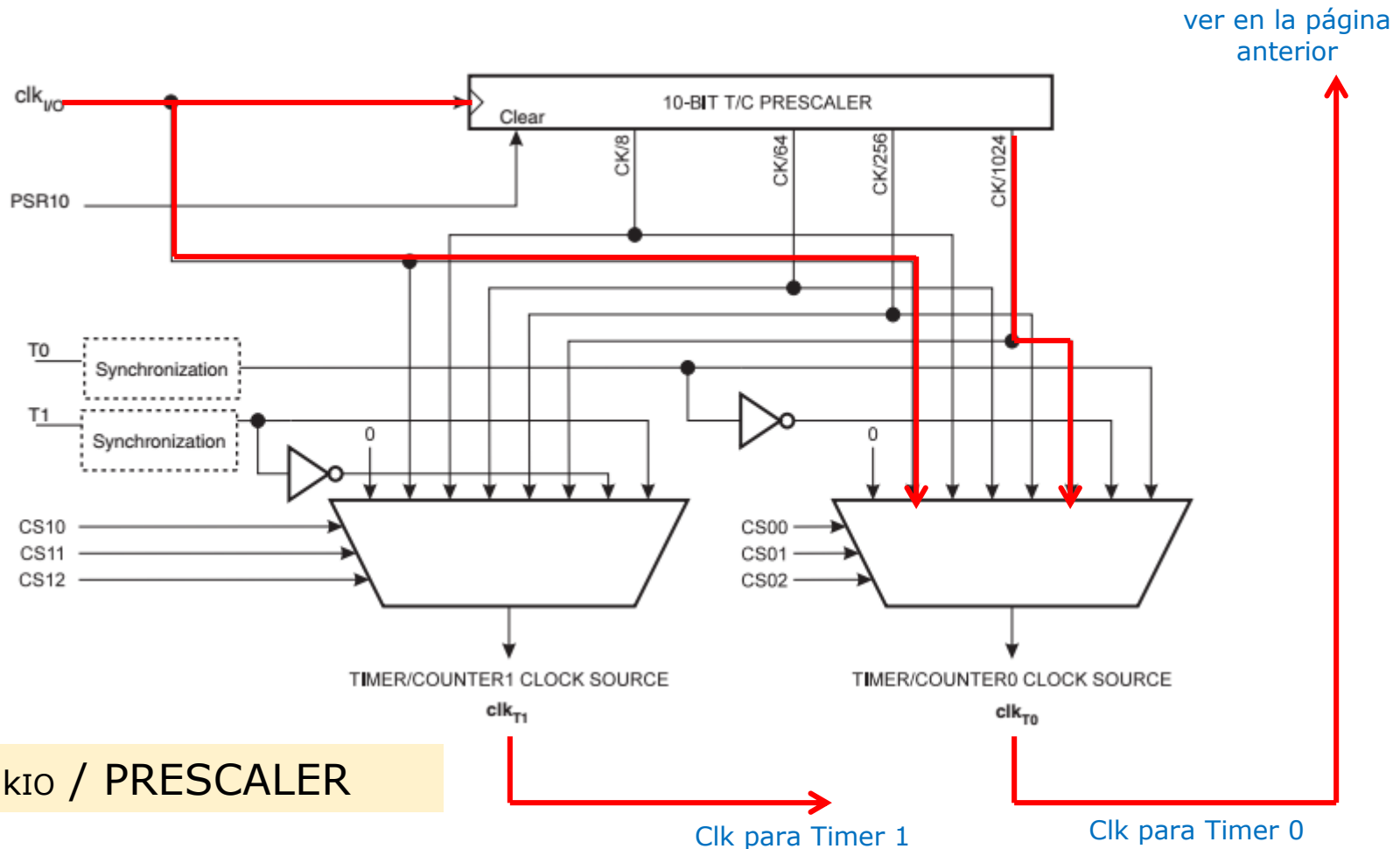
- El sistema se basa en comparar el valor actual del contador TCNT contra un registro que contiene un Valor almacenado (OCRn)



- Cuando el **TCNT** iguala al valor contenido en el registro de comparación **OCR**, se genera un aviso interno (flag de interrupción **OCF**) para que el programa tome alguna acción.
- Además, con un circuito especial (Waveform Generator) se puede modificar el estado del terminal **OCn** cuando se active el flag.
 - Aplicaciones: generación precisa de retardos, generación de señales digitales de frecuencia variable o PWM.

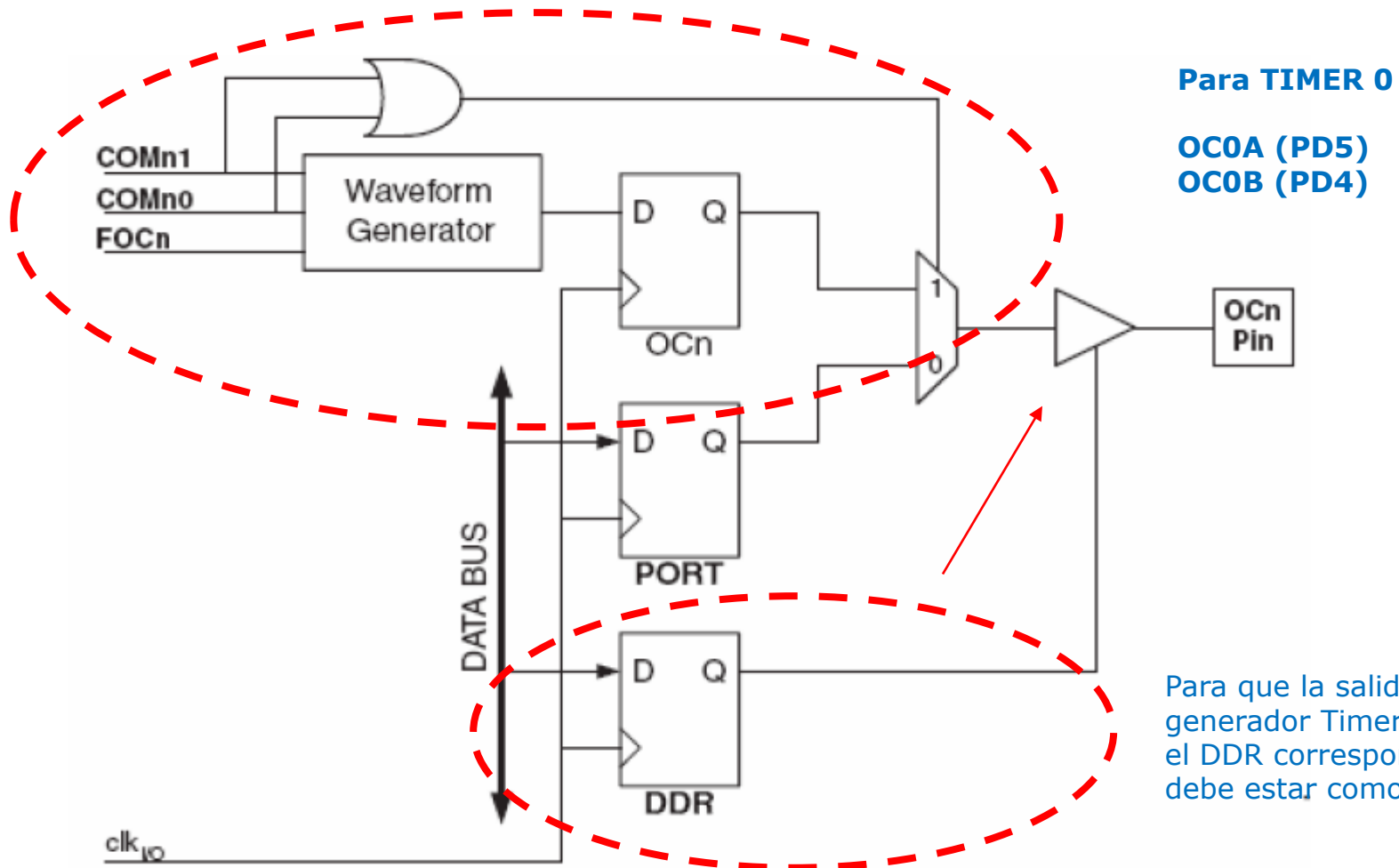
TIMER 0 -Prescalador

- **TIMER 0 y TIMER 1:** El PRESCALADOR permite configurar el reloj de conteo para adecuar la resolución. Además permite configurar las entradas T0, T1 para conteo de pulsos externos.



TIMER 0 – Comparador de salida

- Control de la salida de un terminal I/O



TIMER 0

Registros para su programación

Registros
de configuración

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

Contador

7	6	5	4	3	2	1	0	
TCNT0[7:0]								TCNT0
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

Registros de
comparación

7	6	5	4	3	2	1	0	
OCR0A[7:0]								OCR0A
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
7	6	5	4	3	2	1	0	
OCR0B[7:0]								OCR0B
R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	

Conf. de
interrupciones

7	6	5	4	3	2	1	0	
–	–	–	–	–	OCIE0B	OCIE0A	TOIE0	TIMSK0
R	R	R	R	R	R/W	R/W	R/W	

Banderas de
notificación

7	6	5	4	3	2	1	0	
–	–	–	–	–	OCF0B	OCF0A	TOV0	TIFR0
R	R	R	R	R	R/W	R/W	R/W	

TIMER 0

Registros
de configuración

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–		WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

Table 14-8. Waveform Generation Mode Bit Description

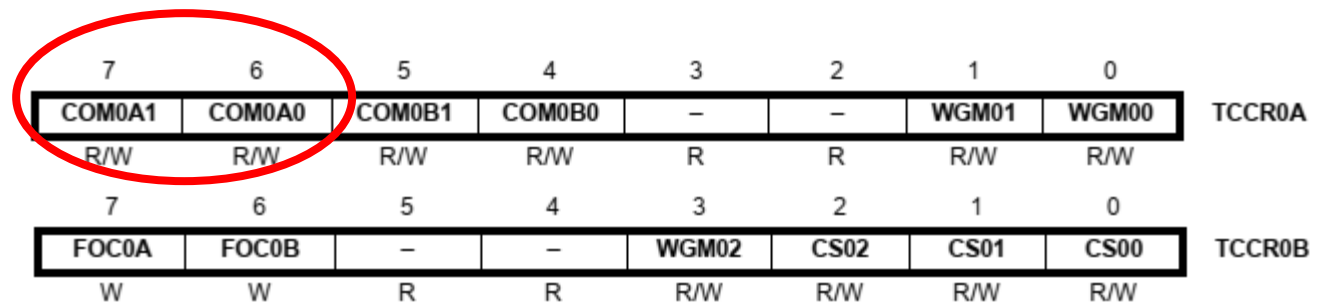
Mode	WGM02	WGM01	WGM00	Timer/Counter Mode of Operation	TOP	Update of OCR _x at	TOV Flag Set on ⁽¹⁾⁽²⁾
0	0	0	0	Normal	0xFF	Immediate	MAX
1	0	0	1	PWM, Phase Correct	0xFF	TOP	BOTTOM
2	0	1	0	CTC	OCRA	Immediate	MAX
3	0	1	1	Fast PWM	0xFF	BOTTOM	MAX
4	1	0	0	Reserved	–	–	–
5	1	0	1	PWM, Phase Correct	OCRA	TOP	BOTTOM
6	1	1	0	Reserved	–	–	–
7	1	1	1	Fast PWM	OCRA	BOTTOM	TOP

Notes: 1. MAX = 0xFF

2. BOTTOM = 0x00

CTC = Clear Timer on Compare match

TIMER 0



Recordemos que hay 2
Generadores de este tipo
Controlados por los comparadores
A y B

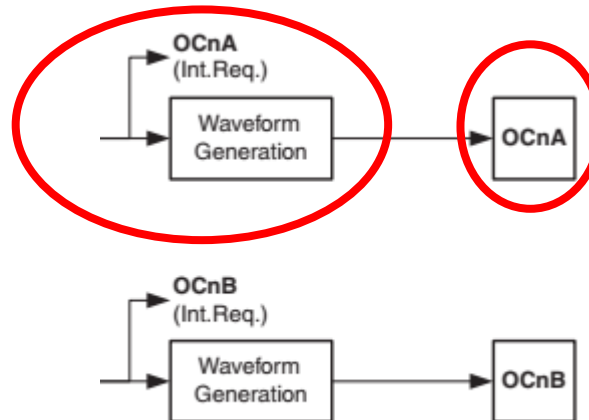


Table 14-2. Compare Output Mode, non-PWM Mode

COM0A1	COM0A0	Description
0	0	Normal port operation, OC0A disconnected.
0	1	Toggle OC0A on Compare Match
1	0	Clear OC0A on Compare Match
1	1	Set OC0A on Compare Match

- Hay otra tabla igual para los bits COM0Bx

No se utiliza la salida Ocnx
Si no se requiere

TIMER 0

7	6	5	4	3	2	1	0	
COM0A1	COM0A0	COM0B1	COM0B0	–	–	WGM01	WGM00	TCCR0A
R/W	R/W	R/W	R/W	R	R	R/W	R/W	
7	6	5	4	3	2	1	0	
FOC0A	FOC0B	–	–	WGM02	CS02	CS01	CS00	TCCR0B
W	W	R	R	R/W	R/W	R/W	R/W	

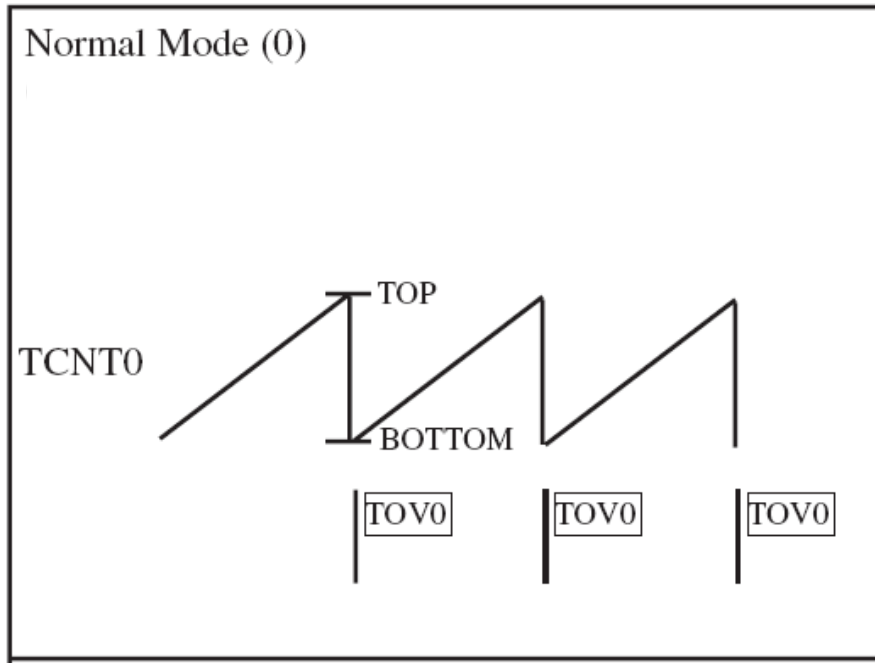
Timer detenido

Table 14-9. Clock Select Bit Description

CS02	CS01	CS00	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	$clk_{I/O}$ /(No prescaling)
0	1	0	$clk_{I/O}/8$ (From prescaler)
0	1	1	$clk_{I/O}/64$ (From prescaler)
1	0	0	$clk_{I/O}/256$ (From prescaler)
1	0	1	$clk_{I/O}/1024$ (From prescaler)
1	1	0	External clock source on T0 pin. Clock on falling edge.
1	1	1	External clock source on T0 pin. Clock on rising edge.

TIMER 0

- Modos de funcionamiento: normal



BOTTOM = 0
TOP = $2^8 - 1 = 255$

TCNT0: contador del Timer0

TOV0: flag de Overflow del Timer0

$$\Rightarrow f_{OVF} = f_{clkTn} / (TOP + 1)$$

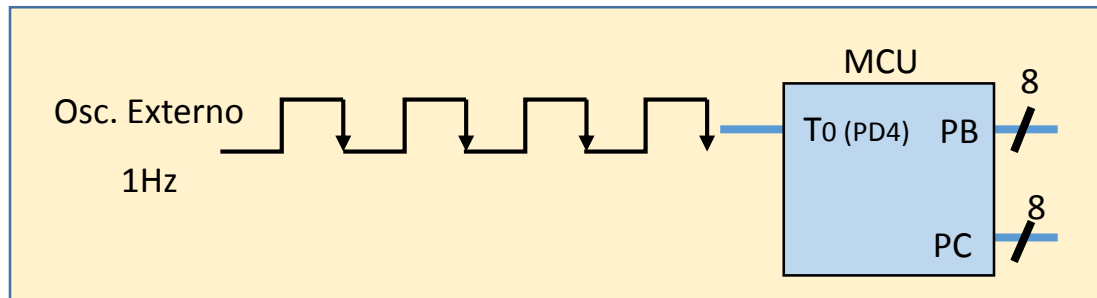
$$f_{clkTn} = f_{clkIO} / \text{PRESCALER}$$

$$f_{clkIO} = 16\text{MHz}$$

TIMER 0

- Ejemplo 1: Utilización como contador de eventos externos

Supongamos que disponemos de un reloj externo de 1Hz conectado al pin T0



Vamos a visualizar en los puertos de salida el conteo de pulsos (16bits) cada 1seg.



TIMER 0

- Ejemplo 1: Utilización como contador de eventos externos

```
#include "avr/io.h"

int main(void) {

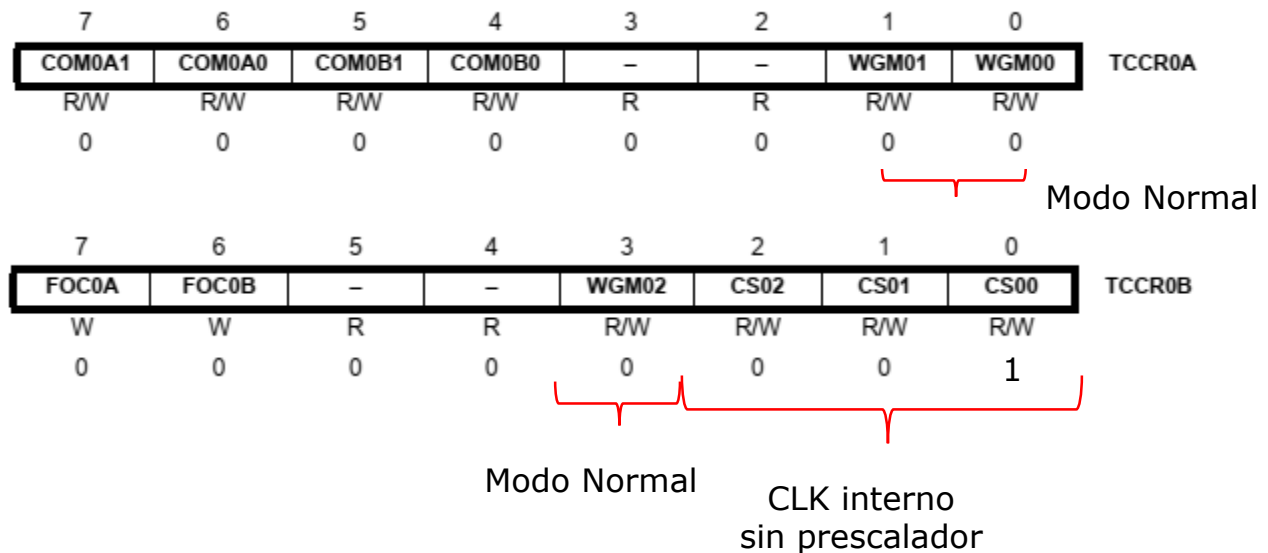
    DDRD&=~(1<<PORTD4);
    PORTD|=(1<<PORTD4);
    DDRB=DDRC=0xFF;
    TCCR0A=0x00; //modo Normal
    TCCR0B=0x06; //external clock T0 pin
    TCNT0=0;

    while(1) {
        do{
            PORTB=TCNT0;
        }while (TIFR0&(1<<TOV0)==0);
        TIFR0|=(1<<TOV0);
        PORTC++;
    }
}
```

Notar que en cada overflow sumo 1 al PORTC, extendiendo la capacidad de conteo del TCNT0

TIMER 0

Ejemplo 2: Configurar modo normal, reloj interno sin prescalador.



- Calcular la resolución de temporización para reloj interno de 2 MHz y 16 MHz:

$$T_{clkT0} = 1 / f_{clkT0} \Rightarrow 500 \text{ ns y } 62.5 \text{ ns respectivamente}$$

- Calcular el tiempo de overflow de dicho contador:

$$T_{OVF} = 2^8 \cdot T_{clkT0} \Rightarrow 128 \text{ us y } 16 \text{ us respectivamente}$$

Se puede observar que a mejor resolución temporal => menor es el tiempo de overflow

TIMER 0

Ejemplo 3: Parpadear un LED en PB5 cada 1 seg utilizando el TIMER 0

```
#include <avr/io.h>
#include <avr/interrupt.h>

// Variable volátil porque se modifica dentro de una interrupción
volatile uint8_t contador_desbordamientos = 0;

int main(void) {
    // 1. Configurar PB5 (Pin 13) como salida
    DDRB |= (1 << DDB5);
    // 2. Configurar Timer 0 en Modo Normal (WGM02:0 = 0)
    // Por defecto, TCCR0A y TCCR0B están en 0, lo cual es Modo Normal.
    // 3. Configurar Prescaler a 1024
    // Según el datasheet: CS02=1, CS01=0, CS00=1
    TCCR0B |= (1 << CS02) | (1 << CS00);
    // 4. Habilitar interrupción por desbordamiento (Timer Overflow Interrupt)
    TIMSK0 |= (1 << TOIE0);
    // 5. Habilitar interrupciones globales
    sei();
    while (1) {
        // El programa principal no hace nada, todo ocurre en la ISR
    }
}
```

TIMER 0

Ejemplo 3: Parpadear un LED en PB5 cada 1 seg utilizando el TIMER 0

Interrupción periódica del TIMER 0



```
// Vector de interrupción para desbordamiento del Timer 0
ISR(TIMERO0_OVF_vect) {
    // Si han pasado ~61 desbordamientos (aprox. 1 segundo)
    if (++contador_desbordamientos >= 61) {
        PORTB ^= (1 << PORTB5); // Cambia el estado del LED (Toggles)
        contador_desbordamientos = 0; // Reinicia el contador
    }
}
```

TIMER 0

- Modos de funcionamiento: CTC

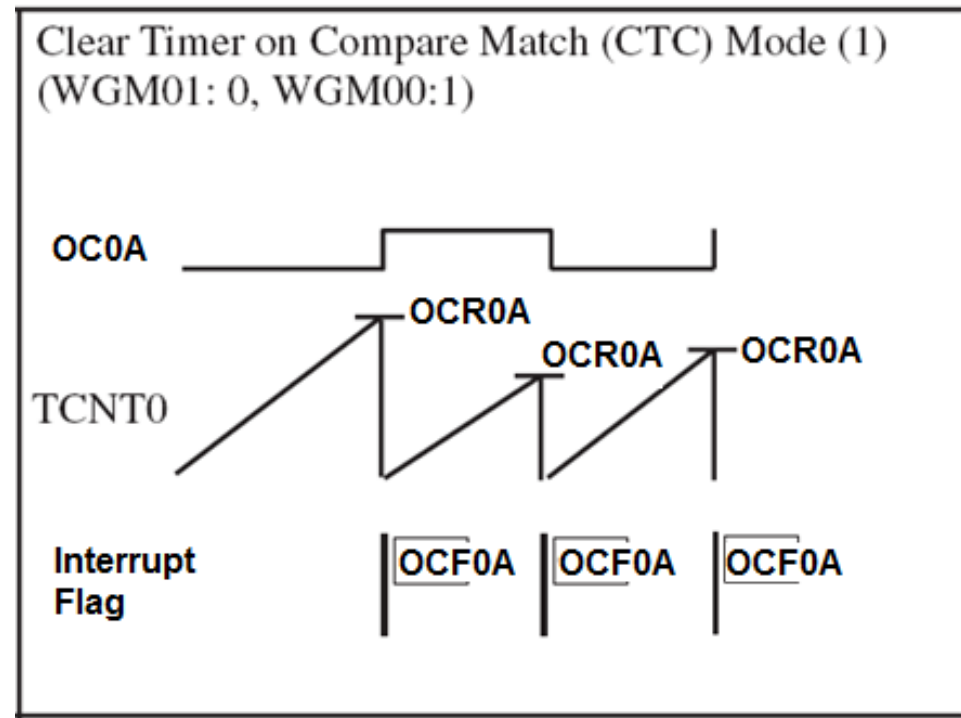
OC0x: Pin salida del periférico

OCF0x: flag de comparación

OCR0x: Registro que contiene el valor a comparar con el TCNT0

Para OCR0x fijo:

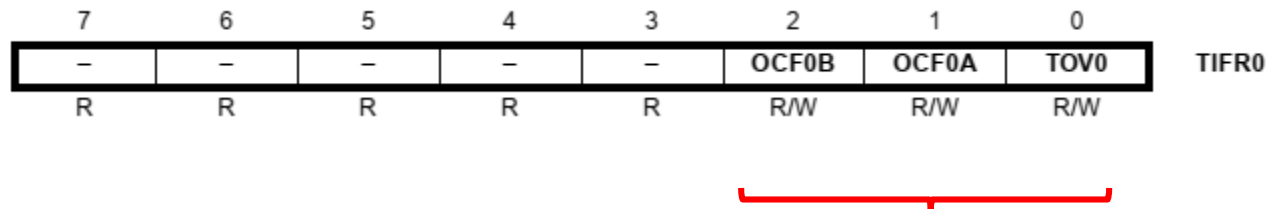
$$\Rightarrow f_{OC0x} = f_{clkTn} / (OCR0x + 1)$$



Esta es la base para generar una interrupción periódica o base de tiempo programable como vimos al principio de la clase

TIMER 0

- Flags o banderas de condición del Timer 0 en el Registro **TIFR0**:



- Activación de los flags:

TOV0 =1 cuando el contador pasa de TOP a 00 (desborde u overflow)

OCF0x =1 cuando el contador iguala al valor del registro OCR0x

- Borrado de los flags:

- OCF0x se borra automáticamente cuando se ejecuta el servicio de interrupción correspondiente.
- En modo polling el programador debe volver a 0 los flags escribiendo un "1" en dichos flags

```
TIFR0 |= (1<<TOV0) ;
```

```
TIFR0 |= (1<<OCF0A) ;
```

TIMER 0

- Ejemplo 4: invertir PB5 utilizando interrupción de comparador A del TIMER 0

```
#include "avr/io.h"
#include "avr/interrupt.h"

int main(void) {

    DDRB |= (1<<PORTB5) ;

    TCCR0A=0x02;    //modo CTC
    TCCR0B=0x01;    //clk prescaler=1
    OCR0A=40;       //módulo
    TIMSK0=(1<<OCIE0A); //int comp A
    sei() ;

    while(1){
        //Tarea de Background
    }

    ISR(TIMERO0_COMPA_vect)
    {
        PORTB^=(1<<PORTB5) ;
    }
```

Interrupción periódica del comparador A

